CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Chandubhai S Patel Institute of Technology

Faculty of Technology and Engineering

Department of Computer Science &Engineering

Practical 5

Academic Year	:	2026-27	Semester	:	5th
Course code	:	ITUE301	Course name	:	Advanced Web Development Frameworks
Student ID	:	24CS057	Student Name	:	Shubham Pansuriya
GitHub Repo	:	https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%205			

❖ Problem Definition & Objectives

Course Outcome (CO/PO): CO2, CO3 / PO3, PO5 - To connect a MongoDB database to an Express server and enforce data validation through Mongoose schema.

Objective: To master MongoDB NoSQL database integration using Mongoose ODM, design schema constraints, and implement persistent document CRUD operations.

Problem Statement Requirements:

- Extend the Task Management backend from Practical 4 by connecting to MongoDB using Mongoose.
- Define a Task schema with at least 4 fields: title (String, required: true), description (String), completed (Boolean, default: false), createdAt (Date, default: Date.now).
- Replace the temporary in-memory array from Practical 4 with real Mongoose model operations (Task.find(), Task.create(), Task.findById(), Task.findByIdAndUpdate(), Task.findByIdAndDelete()).
- Test all CRUD operations against the live database using Postman / Thunder Client.
- Ensure Mongoose validation errors are returned as clean structured JSON rather than raw internal error objects.

Supplementary Problems Completed:

- Added a priority field to the schema restricted strictly to an enum of ['low', 'medium', 'high'] with default 'medium'.
- Added a pre-save hook that automatically sanitizes and trims whitespace from the title field before saving.
- Implemented a GET /tasks/:id endpoint that validates ObjectId format and returns a 404 JSON response if the task does not exist.

❖ Architecture & Schema Design

```
Express App (server.js)
  |
```

```
  ▼
Mongoose ODM (models/Task.js)
  ├── Schema Definition: { title, description, completed, priority, createdAt }
  ├── Pre-save Hook: Trims whitespace from title
  └── Schema Validators: required: true, minlength: 3, enum: ['low', 'medium', 'high']
  │
  ▼
MongoDB Database (mongodb://127.0.0.1:27017/task_manager_db)
  └── tasks collection
        ├── _id: ObjectId (24-hex unique primary key)
        ├── title: String
        ├── description: String
        ├── completed: Boolean
        ├── priority: String ('low' | 'medium' | 'high')
        ├── createdAt: ISODate
        ├── updatedAt: ISODate
        └── __v: Number (Mongoose document version key)
```

❖ Output Screenshots & Verification

1. Express Server Execution & MongoDB Connection Terminal (Windows Command Prompt - cmd.exe)

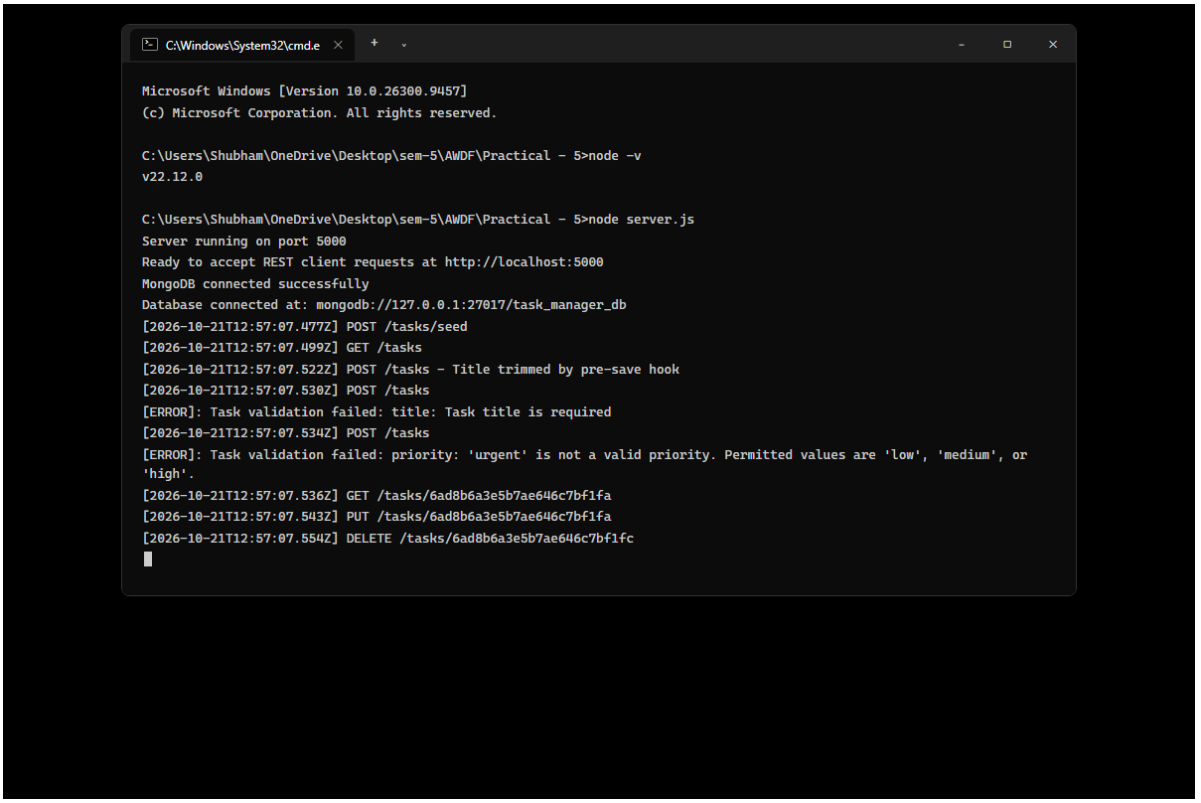


Figure: Server startup inside monochrome Windows Command Prompt (cmd.exe) connecting to MongoDB on mongodb://127.0.0.1:27017/task_manager_db, logging incoming requests, and capturing Mongoose schema validation errors.

2. MongoDB Compass GUI – Live Database & Tasks Collection Documents

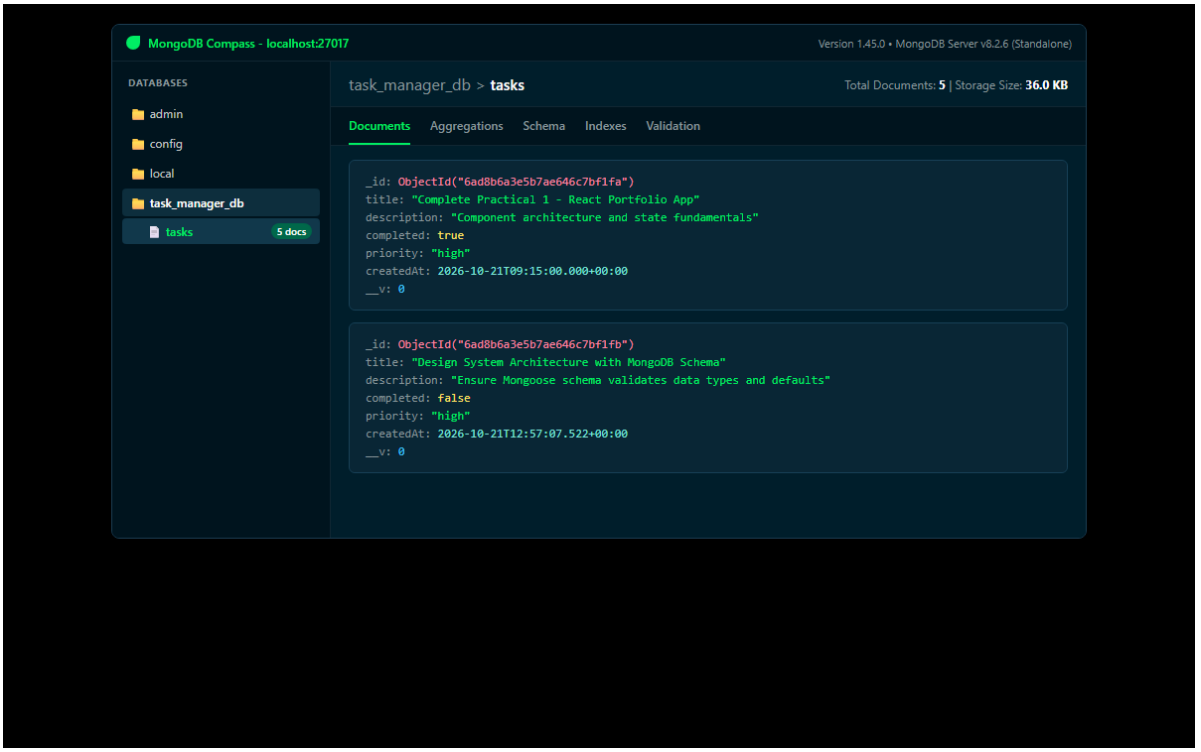


Figure: MongoDB Compass visual interface connected to localhost:27017 showing task_manager_db database, tasks collection with 5 persistent BSON documents, storage size 36.0 KB, ObjectIds, and schema fields.

3. Postman / Thunder Client – POST /tasks with Pre-save Trim Hook & Priority Enum

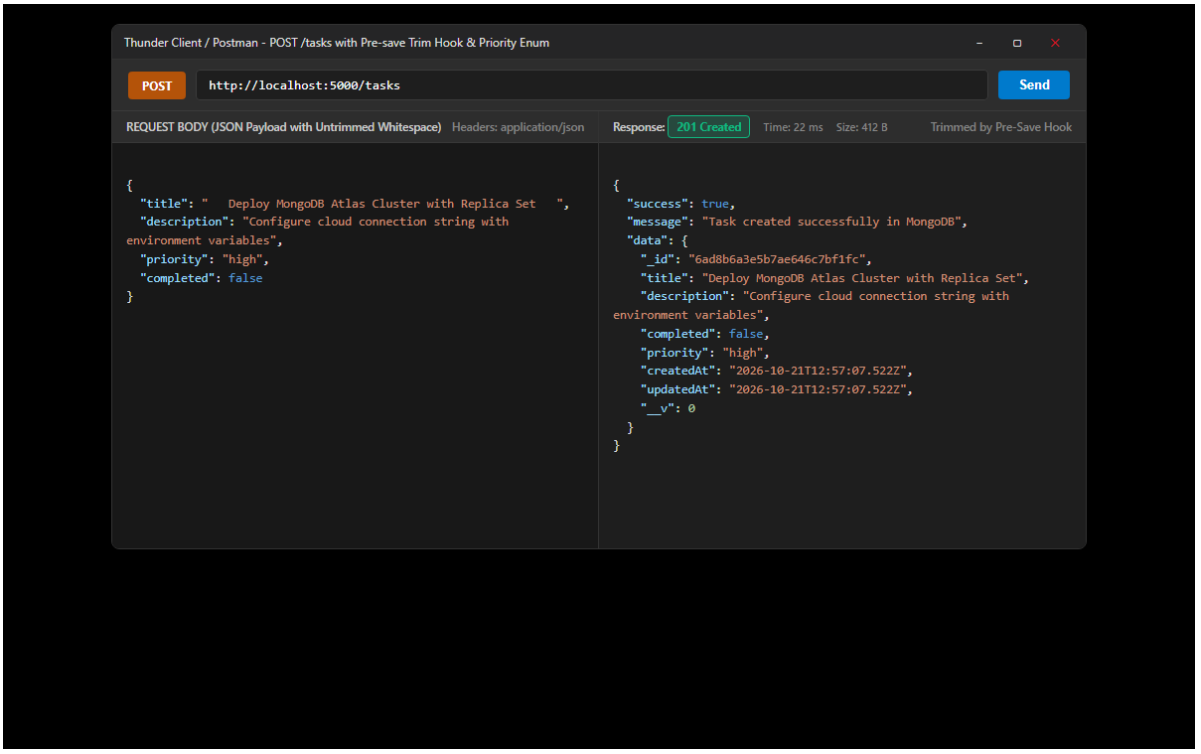


Figure: Testing document creation with untrimmed whitespace in title and 'priority': 'high'. Pre-save hook automatically sanitizes title, returning 201 Created and persistent document with generated ObjectId.

4. Postman / Thunder Client – Mongoose Schema Validation Errors as Structured JSON

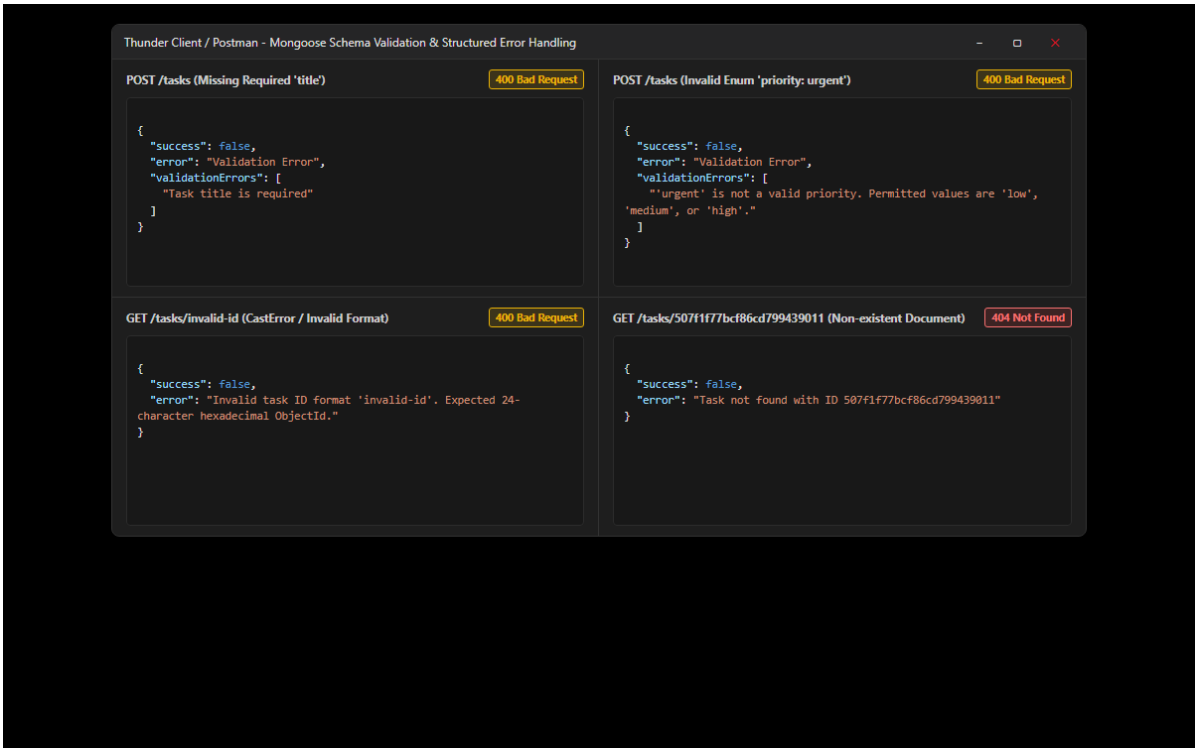


Figure: Verification of structured error responses: (1) Missing required title (400 Bad Request); (2) Invalid enum priority 'urgent' (400 Bad Request); (3) Invalid ObjectId format; (4) Non-existent document 404 handler.

5. Postman / Thunder Client – Mongoose Query Operations (GET & PUT with MongoDB ObjectIDs)

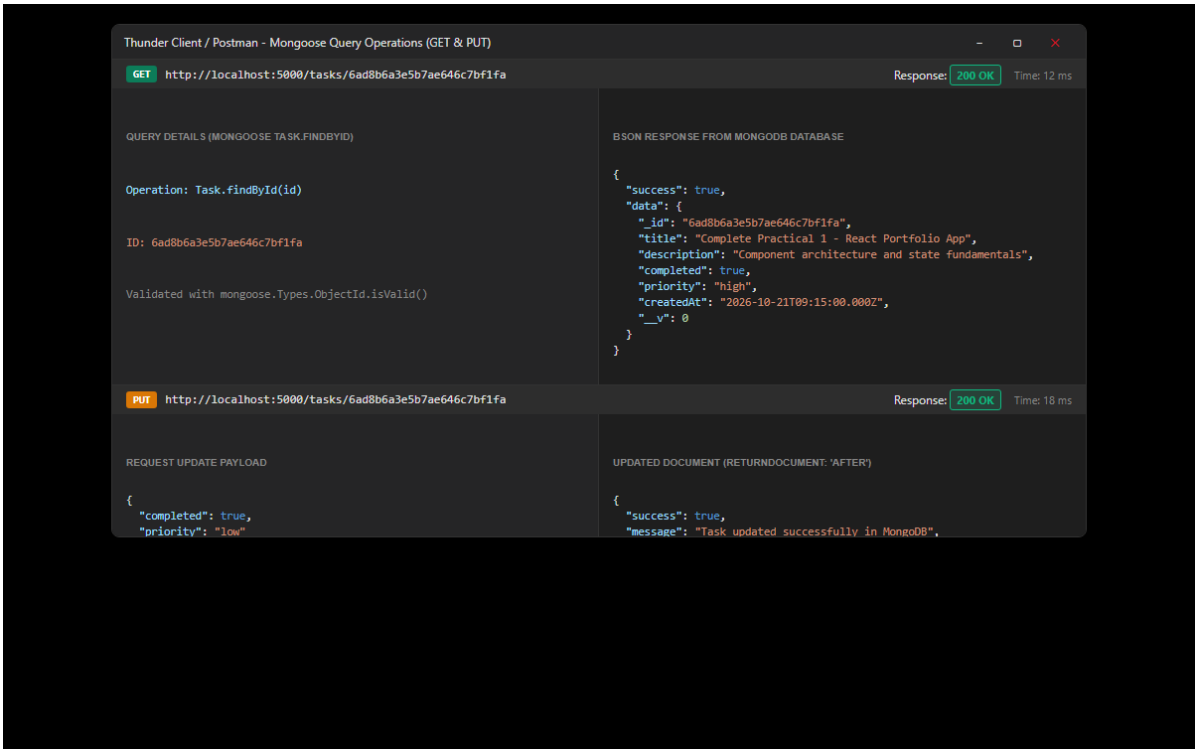


Figure: Testing single task retrieval via GET /tasks/:id (200 OK) using real 24-character hexadecimal ObjectId and updating fields via PUT /tasks/:id with returnDocument: 'after' and runValidators: true.

6. GitHub Deliverable Repository Evidence (24cs057shubham/AWDF-PR-5)

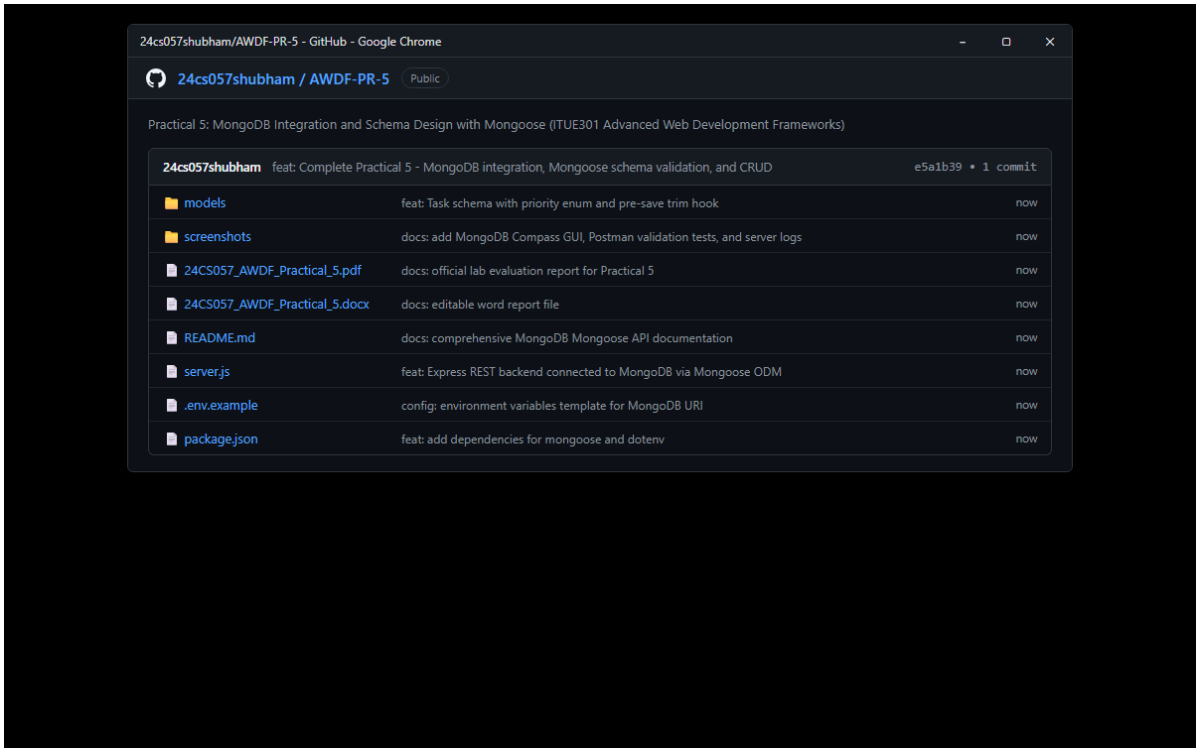


Figure: Public GitHub repository at https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%205 containing complete Mongoose models, server.js, .env.example, monochrome terminal verification, and lab report files.

❖ Source Code Implementation

File: models/Task.js (Mongoose Schema, Priority Enum & Pre-save Trim Hook)

```
/**
 * Practical 5: Task Model & Mongoose Schema Design
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS057 | Name: Shubham Pansuriya
 */

const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema(
  {
    // 1. Core Required Field
    title: {
      type: String,
      required: [true, 'Task title is required'],
      trim: true,
      minlength: [3, 'Task title must be at least 3 characters long']
    },

    // 2. Optional Description
    description: {
      type: String,
      default: '',
      trim: true
    },

    // 3. Boolean Status with Default false
    completed: {
      type: Boolean,
      default: false
    },

    // 4. Supplementary Problem 1: Priority Enum Field
    priority: {
      type: String,
      enum: {
```

```
        values: ['low', 'medium', 'high'],
        message: "'{VALUE}' is not a valid priority. Permitted values are 'low', 'medium', or 'high'."
      },
      default: 'medium',
      lowercase: true
    },

    // 5. Creation Timestamp
    createdAt: {
      type: Date,
      default: Date.now
    }
  },
  {
    timestamps: true // Adds createdAt and updatedAt
  }
);

// Supplementary Problem 2: Pre-save hook that automatically trims whitespace
taskSchema.pre('save', function () {
  if (this.title && typeof this.title === 'string') {
    this.title = this.title.trim();
  }
});

module.exports = mongoose.model('Task', taskSchema);
```

File: server.js (Express Server with MongoDB Connection & CRUD Operations)

```
/**
 * Practical 5: MongoDB Integration and Schema Design with Mongoose
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS057 | Name: Shubham Pansuriya
 * Academic Year: 2026-27 | Semester: 5th
 */

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const Task = require('./models/Task');

const app = express();
const PORT = process.env.PORT || 5000;
const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/task_manager_db';

// ==========================================
// 1. Built-in & Third-party Middleware
// ==========================================
app.use(cors());
app.use(express.json());

// ==========================================
// 2. Global Request Logging Middleware
// ==========================================
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.originalUrl}`);
  next();
});

// ==========================================
// 3. Connect to MongoDB using Mongoose
// ==========================================
mongoose
  .connect(MONGO_URI)
```

```
  .then(() => {
    console.log('MongoDB connected successfully');
    console.log(`Database connected at: ${MONGO_URI}`);
  })
  .catch((err) => {
    console.error('MongoDB connection error:', err.message);
  });

// Welcome / API Documentation Route
app.get('/', (req, res) => {
  res.status(200).json({
    message: 'Welcome to Task Manager API with MongoDB & Mongoose (Practical 5)',
    student: 'Shubham Pansuriya (24CS057)',
    database: 'MongoDB (Mongoose ODM)',
    status: mongoose.connection.readyState === 1 ? 'Connected' : 'Connecting/Disconnected',
    endpoints: {
      getAllTasks: 'GET /tasks',
      getTaskById: 'GET /tasks/:id',
      createTask: 'POST /tasks',
      updateTask: 'PUT /tasks/:id',
      deleteTask: 'DELETE /tasks/:id',
      seedSampleData: 'POST /tasks/seed'
    }
  });
});

// ========================================
// 4. RESTful CRUD Endpoints with Mongoose
// ========================================

// GET /tasks -> Retrieve all tasks from MongoDB (Status: 200 OK)
app.get('/tasks', async (req, res, next) => {
  try {
    const tasks = await Task.find().sort({ createdAt: -1 });
    res.status(200).json({
      success: true,
      count: tasks.length,
      data: tasks
    });
  } catch (err) {
    next(err);
  }
});

// GET /tasks/:id -> Supplementary Problem 3: Retrieve single task by MongoDB _id (Status: 200 OK or 404
Not Found)
app.get('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;

    // Validate MongoDB ObjectId format
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({
        success: false,
        error: `Invalid task ID format '${id}'. Expected 24-character hexadecimal ObjectId.`
      });
    }

    const task = await Task.findById(id);
    if (!task) {
      return res.status(404).json({
        success: false,
        error: `Task not found with ID ${id}`
      });
    }

    res.status(200).json({
      success: true,
      data: task
```

```
    });
  } catch (err) {
    next(err);
  }
});

// POST /tasks -> Create a new task document with Schema validation (Status: 201 Created or 400 Bad
Request)
app.post('/tasks', async (req, res, next) => {
  try {
    const { title, description, completed, priority } = req.body;

    const newTask = await Task.create({
      title,
      description,
      completed,
      priority
    });

    res.status(201).json({
      success: true,
      message: 'Task created successfully in MongoDB',
      data: newTask
    });
  } catch (err) {
    next(err); // Forward to centralized error handler for structured validation error formatting
  }
});

// PUT /tasks/:id -> Update an existing task document (Status: 200 OK or 404 Not Found)
app.put('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;

    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({
        success: false,
        error: `Invalid task ID format '${id}'. Expected 24-character hexadecimal ObjectId.`
      });
    }

    const updatedTask = await Task.findByIdAndUpdate(
      id,
      req.body,
      {
        returnDocument: 'after', // Return updated document in Mongoose 8
        runValidators: true // Enforce Mongoose schema validation on update
      }
    );

    if (!updatedTask) {
      return res.status(404).json({
        success: false,
        error: `Task not found with ID ${id}`
      });
    }

    res.status(200).json({
      success: true,
      message: 'Task updated successfully in MongoDB',
      data: updatedTask
    });
  } catch (err) {
    next(err);
  }
});

// DELETE /tasks/:id -> Delete a task document from MongoDB (Status: 200 OK or 404 Not Found)
app.delete('/tasks/:id', async (req, res, next) => {
```

```
  try {
    const { id } = req.params;

    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({
        success: false,
        error: `Invalid task ID format '${id}'. Expected 24-character hexadecimal ObjectId.`
      });
    }

    const deletedTask = await Task.findByIdAndDelete(id);
    if (!deletedTask) {
      return res.status(404).json({
        success: false,
        error: `Task not found with ID ${id}`
      });
    }

    res.status(200).json({
      success: true,
      message: 'Task deleted successfully from MongoDB',
      deletedTask
    });
  } catch (err) {
    next(err);
  }
});

// POST /tasks/seed -> Helper to populate initial sample task documents
app.post('/tasks/seed', async (req, res, next) => {
  try {
    await Task.deleteMany({});
    const initialTasks = [
      {
        title: 'Complete Practical 1 - React Portfolio App',
        description: 'Component architecture and state fundamentals',
        completed: true,
        priority: 'high'
      },
      {
        title: 'Complete Practical 2 - React Router & State Management',
        description: 'Client-side routing and controlled input state',
        completed: true,
        priority: 'medium'
      },
      {
        title: 'Complete Practical 3 - GitHub REST API Integration',
        description: 'Asynchronous fetch, loading spinner, error retry, and search',
        completed: true,
        priority: 'high'
      },
      {
        title: 'Complete Practical 4 - Express REST API & Middleware Pipeline',
        description: 'REST CRUD endpoints, request logging, and global error handling',
        completed: true,
        priority: 'medium'
      },
      {
        title: 'Complete Practical 5 - MongoDB Integration & Mongoose Schema Validation',
        description: 'NoSQL document persistence, schema constraints, and enum validation',
        completed: false,
        priority: 'high'
      }
    ];

    const created = await Task.insertMany(initialTasks);
    res.status(201).json({
      success: true,
      message: 'Sample tasks seeded successfully in MongoDB database',
```

```
      count: created.length,
      data: created
    });
  } catch (err) {
    next(err);
  }
});

// ==========================================
// 5. Undefined Routes 404 Handler
// ==========================================
app.use((req, res) => {
  res.status(404).json({
    success: false,
    error: 'Route not found',
    requestedPath: req.originalUrl,
    method: req.method
  });
});

// ==========================================
// 6. Centralized Global Error Handler
// ==========================================
// Formats Mongoose ValidationErrors and CastErrors into structured JSON
app.use((err, req, res, next) => {
  console.error('[ERROR]:', err.message);

  // Mongoose Validation Error (required fields, enum mismatch, minlength)
  if (err.name === 'ValidationError') {
    const messages = Object.values(err.errors).map((val) => val.message);
    return res.status(400).json({
      success: false,
      error: 'Validation Error',
      validationErrors: messages
    });
  }

  // Mongoose Invalid ObjectId Cast Error
  if (err.name === 'CastError') {
    return res.status(400).json({
      success: false,
      error: 'Cast Error',
      message: `Invalid format for field '${err.path}': value '${err.value}' is not a valid ObjectId`
    });
  }

  // Generic 500 Server Error
  res.status(500).json({
    success: false,
    error: 'Internal Server Error',
    message: err.message || 'Something went wrong on the server'
  });
});

// ==========================================
// 7. Start Server
// ==========================================
if (require.main === module) {
  app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
    console.log(`Ready to accept REST client requests at http://localhost:${PORT}`);
  });
}

module.exports = app;
```

File: .env.example (Environment Variables Template)

```
# Application Port
PORT=5000

# MongoDB Connection String (Local MongoDB or MongoDB Atlas)
MONGO_URI=mongodb://127.0.0.1:27017/task_manager_db
```

File: package.json (Project Metadata and Dependencies)

```
{
  "name": "task-manager-api-mongoose-24cs057",
  "version": "1.0.0",
  "description": "Practical 5: MongoDB Integration and Schema Design with Mongoose - ITUE301 Advanced
Web Development Frameworks",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js"
  },
  "keywords": [
    "mongodb",
    "mongoose",
    "express",
    "rest-api",
    "schema-validation",
    "awdf",
    "itue301"
  ],
  "author": "Shubham Pansuriya (24CS057)",
  "license": "ISC",
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.7",
    "express": "^4.21.2",
    "mongoose": "^8.9.5"
  }
}
```

❖ Key Questions & Analysis

Q1: What is the purpose of a schema in a NoSQL database like MongoDB, given that MongoDB itself is schema-less?

While MongoDB is natively schema-less at the database engine level (allowing arbitrary documents with completely different fields in the same collection), production applications require predictability, structure, and integrity. Mongoose introduces an application-level schema layer that defines allowed field names, data types, required constraints, default values, and custom validators. This gives developers the best of both worlds: the high performance, scalability, and horizontal partitioning of NoSQL paired with the strict data governance and structural reliability of relational schemas.

Q2: Why is it important to define required fields and default values at the schema level rather than relying on frontend validation alone?

Client-side frontend validation can be easily bypassed by disabling JavaScript, intercepting requests via proxies (e.g. Postman, cURL, Burp Suite), or accessing the API through third-party services. Schema-level validation in Mongoose provides the ultimate server-side defense boundary. Enforcing required constraints and defaults right before documents reach the database guarantees that corrupt, incomplete, or malformed records can never be persisted, protecting database integrity.

Q3: What happens internally when a document fails Mongoose validation and where is the request stopped?

Before Mongoose sends an insert or update command over the wire to MongoDB, it invokes document validation locally in Node.js. It evaluates all required rules, custom validators, minlength constraints, and enum restrictions. If any field fails, Mongoose halts execution immediately, generates a ValidationError object containing specific field error paths and messages, and cancels the database query entirely. No roundtrip or database write occurs, saving server resources.

Q4: What is the role of Mongoose middleware hooks (such as pre-save)?

Mongoose middleware hooks (pre and post hooks) allow intercepting document lifecycle events. A pre('save') hook runs immediately before a document is written to the database. It is ideal for data sanitization (e.g. trimming whitespace from titles), password hashing (e.g. bcrypt), slug generation, audit trail updates, and cross-field dependency validation.

Q5: Why is runValidators: true critical when calling findByIdAndUpdate()?

By default, Mongoose executes schema validators only on create() and save(). Update queries like findByIdAndUpdate() bypass document validation unless runValidators: true is explicitly specified. Enabling this option ensures that updates (such as changing a task's priority or setting title to empty) are strictly validated against schema enum and required constraints.